CS 486/686
Neural Networks and Learned Embeddings

Yuntian Deng

Lecture 18

From fixed features to learned vector representations

Recorded lecture (asynchronous)

🎥 This lecture is **pre-recorded** - I'm away at **ICML** this week, so there's no in-person class today. Watch at your own pace.

📅 **Due today (Thu Jul 9):** Assignment 2. Chat 9 is now out.

💬 Questions? Post on **Piazza** - the TAs are available, and I'll follow up when I'm back.

The problem with fixed features

L17 assumed the useful input vector already exists.

pixels

Where is the cat ear?

word IDs

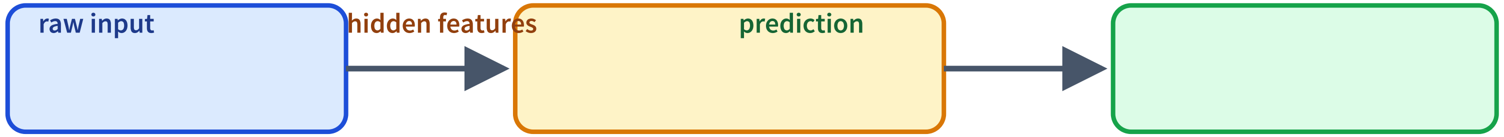
Why is dog close to puppy?

screenshots

Which region is a button?

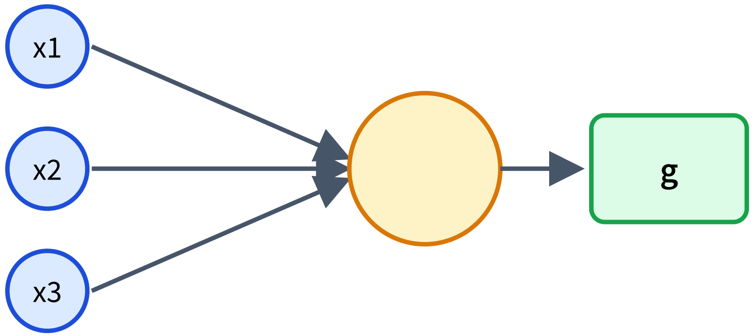
Modern ML learns the features too.

Neural nets learn representations



The hidden layer is not just computation: it is a learned view of the input.

A neuron is a feature detector



$$z = w^\top x + b$$

$$a = g(z)$$

Weights decide what pattern to detect.

Why add a nonlinearity?

Stacking linear layers is still linear.

$$W_2(W_1x) = (W_2W_1)x$$

ReLU

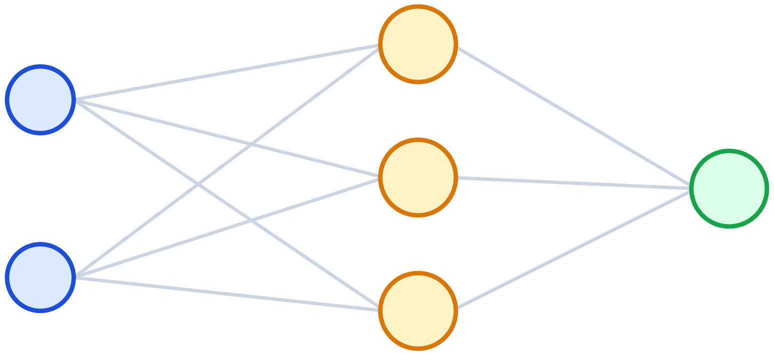
$$g(x) = \max(0, x)$$

GELU / SiLU

smooth modern defaults

Nonlinearity lets layers bend the representation.

One hidden layer = learned feature map

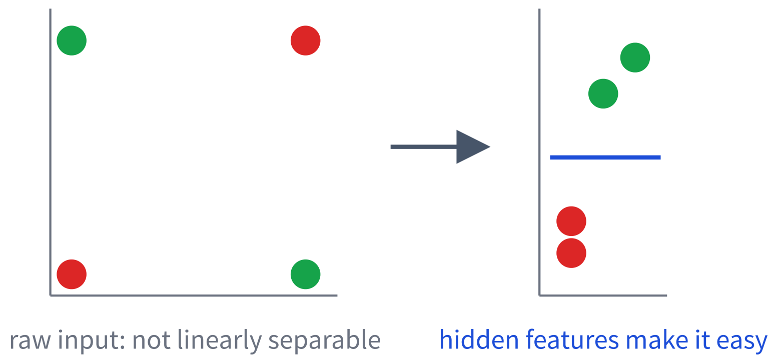


Each hidden unit learns a detector.

The output layer combines detectors.

This is how fixed features become learned features.

XOR: why hidden features help



A hidden layer can transform the space.

Then a simple output layer can solve the task.

An MLP is a differentiable program

$$h = g(W_1 x + b_1)$$

$$\hat{y} = \mathrm{softmax}(W_2 h + b_2)$$

Every operation is differentiable, so L17's training loop applies unchanged.

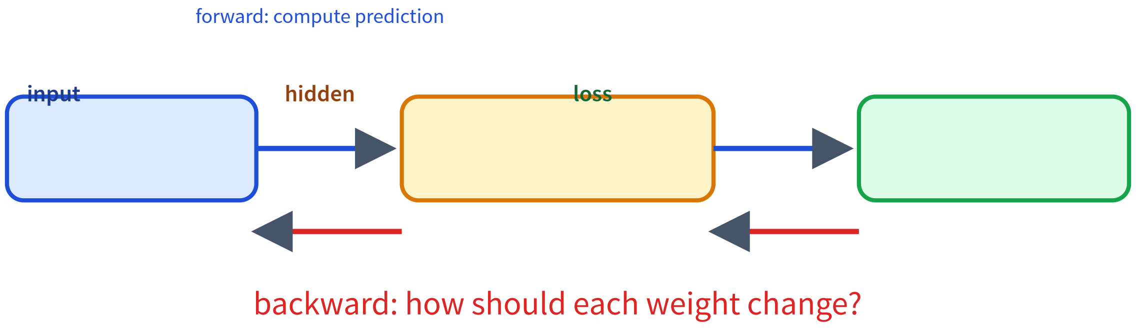
The code barely changes

```
model = nn.Sequential(
    nn.Linear(d_in, 64),
    nn.ReLU(),
    nn.Linear(64, n_classes),
)

loss = loss_fn(model(x), y)
loss.backward()
optimizer.step()
```

The function is richer; the training pattern is the same.

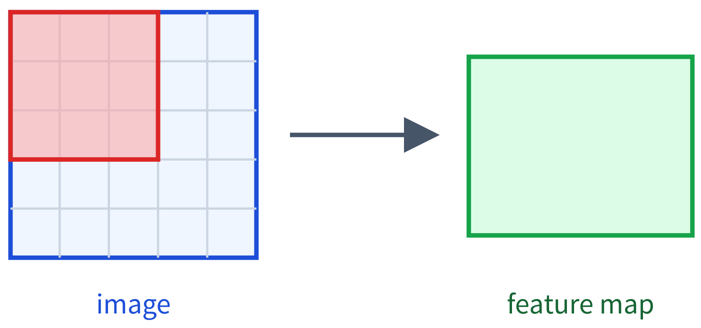
Backpropagation assigns credit



What do hidden layers learn?

Deep models compose simple features into useful abstractions.

CNNs learn visual features



Local filters detect patterns in patches.

Weight sharing detects the same pattern anywhere.

Later: VLMs use image encoders to turn images into vectors/tokens.

Word IDs have no geometry

A raw ID is arbitrary:

$$\texttt{cat} = 17, \texttt{dog} = 932$$

An embedding gives each item a vector:

$$\mathbf{Embed}[\mathbf{cat}] \in \mathbb{R}^d$$

Now similarity can live in vector space.

An embedding table is just parameters

item	learned vector
cat	[0.22, -0.71, 0.10, ...]
dog	[0.19, -0.66, 0.12, ...]
car	[-0.80, 0.31, -0.40, ...]

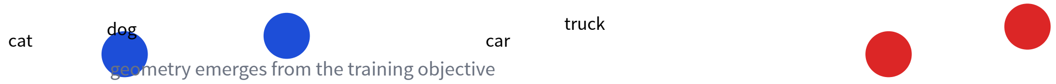
```
emb = nn.Embedding(
    num_embeddings=vocab_size,
    embedding_dim=128,
)
```

Gradient descent updates these vectors during training.

How does an embedding learn?

`the cat sat on the ___`

If the model should predict **mat**, gradients update the embeddings that helped make the prediction.



Embedding geometry is useful, not magic

useful

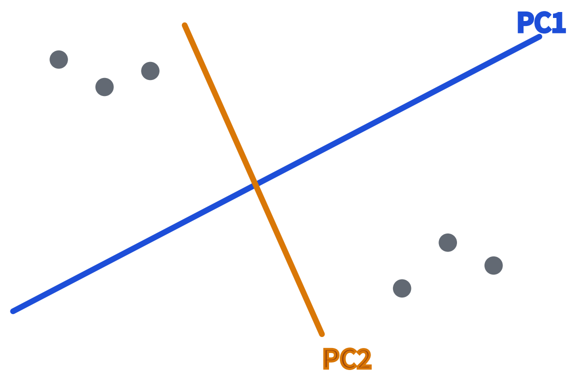
nearest neighbors, retrieval, clustering

dangerous

2D plots can overstate structure

Distances mean what the training objective made them mean.

Inspect embeddings by projecting them

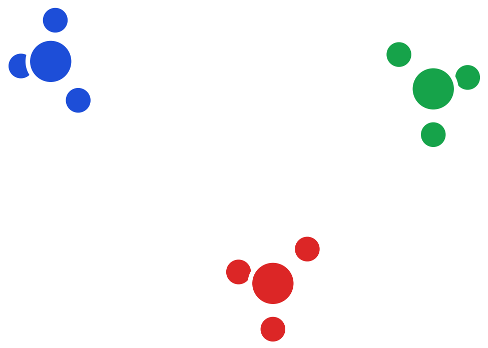


PCA: linear view of high-dimensional vectors.

t-SNE/UMAP: nonlinear visualization.

Use plots to ask questions, not as proof.

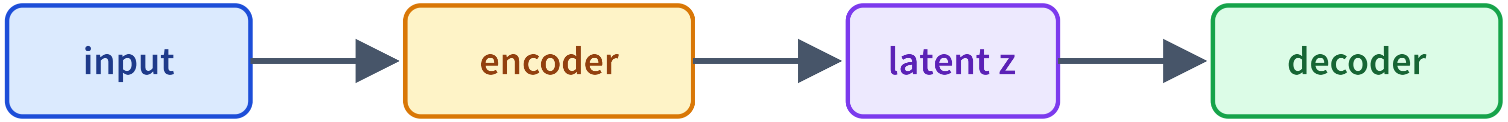
Cluster embeddings with k-means



After embeddings exist, k-means becomes practical.

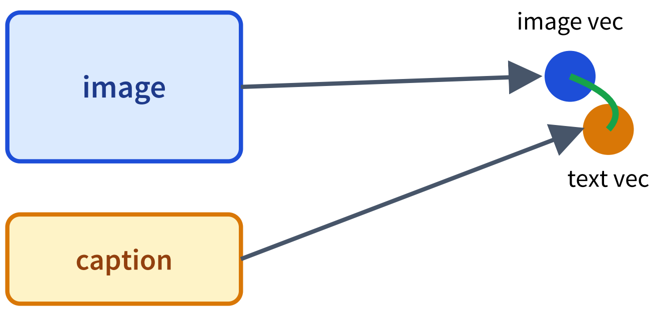
- inspect datasets
- find groups or duplicates
- build codebook intuition

Autoencoders learn compression



Latent diffusion will generate in a compressed latent space.

Contrastive learning aligns modalities



Matched image/text pairs are pulled together.

Mismatched pairs are pushed apart.

This is the bridge to VLMs and retrieval.

Why this matters for LLMs

tokens

become embeddings

attention

moves information between embeddings

transformers

scale this up

Recap

✓ Neural nets learn **features**, not just predictions.

✓ Backprop assigns credit through the graph.

✓ **Embeddings** are learned vectors.

✓ PCA, t-SNE, and k-means help inspect embeddings.

✓ Autoencoders and CLIP prepare us for diffusion and VLMs.

Next question

A sentence is a sequence of embeddings.

How should each word look at the other words?

L19: sequence models, attention, and transformers.